UNIT 8 : ADVANCED FORECASTING METHODS

BY

KAPIL RATHOR

AI DEPARTMENT

Complex seasonality

- So far, we have mostly considered relatively simple seasonal patterns such as quarterly and monthly data.
- However, higher frequency time series often exhibit more complicated seasonal patterns.
- For example, daily data may have a weekly pattern as well as an annual pattern.
- Hourly data usually has three types of seasonality: a daily pattern, a weekly pattern, and an annual pattern.
- Even weekly data can be challenging to forecast as there are not a whole number of weeks in a year, so the annual pattern has a seasonal period of $365.25/7 \approx 52.179$ on average.
- Most of the methods we have considered so far are unable to deal with these seasonal complexities.

Complex seasonality

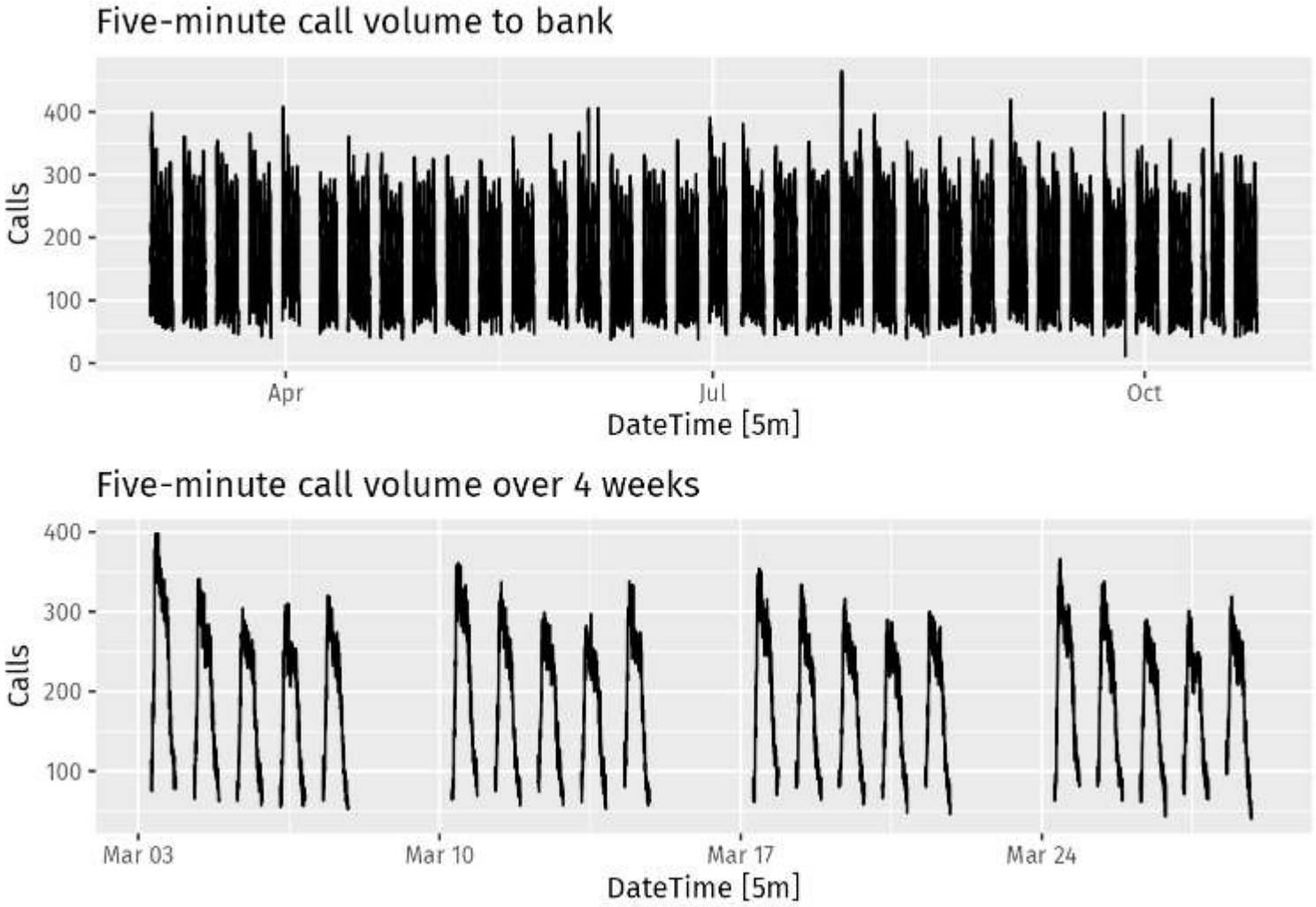
- We don’t necessarily want to include all the possible seasonal periods in our models — just the ones that are likely to be present in the data.

- For example, if we have only 180 days of data, we may ignore the annual seasonality.

- If the data are measurements of a natural phenomenon (e.g., temperature), we can probably safely ignore any weekly seasonality.

Example

- Figure shows the number of calls to a North American commercial bank per 5-minute interval between 7:00am and 9:05pm each weekday over a 33-week period.
- The lower panel shows the first four weeks of the same time series. There is a strong daily seasonal pattern with period 169 (there are 169 5-minute intervals per day), and a weak weekly seasonal pattern with period 169×5=845.
- (Call volumes on Mondays tend to be higher than the rest of the week.)
- If a longer series of data were available, we may also have observed an annual seasonal pattern.

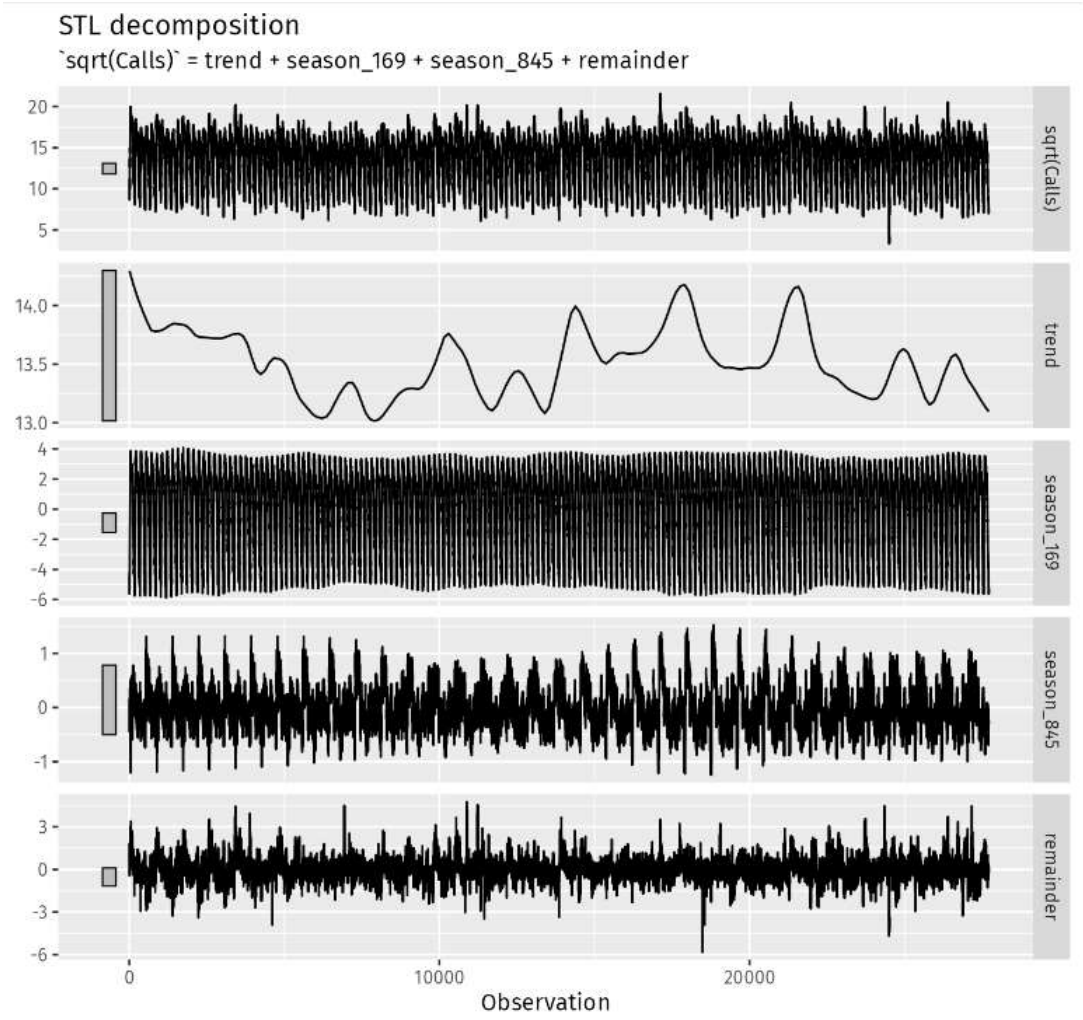


STL with multiple seasonal periods

The STL() function is designed to deal with multiple seasonality. It will return multiple seasonal components, as well as a trend and remainder component.

In this case, we need to re-index the tsibble to avoid the missing values, and then explicitly give the seasonal periods.

There are two seasonal patterns shown, one for the time of day (the third panel), and one for the time of week (the fourth panel). To properly interpret this graph, it is important to notice the vertical scales. In this case, the trend and the weekly seasonality have wider bars (and therefore relatively narrower ranges) compared to the other components, because there is little trend seen in the data, and the weekly seasonality is weak.



Dynamic harmonic regression with multiple seasonal periods：Fourier series

Periodic seasonality can be handled using pairs of Fourier terms:

$$s_k(t) = \sin\left(\frac{2\pi kt}{m}\right) \qquad c_k(t) = \cos\left(\frac{2\pi kt}{m}\right)$$

$$y_t = a + bt + \sum_{k=1}^{K} [\alpha_k s_k(t) + \beta_k c_k(t)] + \varepsilon_t$$

- Every periodic function can be approximated by sums of sin and cos terms for large enough K.
- Choose K by minimizing AICc.
- Called “harmonic regression”

Periodic seasonality can be handled using pairs of Fourier terms:

$$s_k(t) = \sin\left(\frac{2\pi kt}{m}\right) \qquad c_k(t) = \cos\left(\frac{2\pi kt}{m}\right)$$

$$y_t = a + bt + \sum_{k=1}^{K} [\alpha_k s_k(t) + \beta_k c_k(t)] + \varepsilon_t$$

- Every periodic function can be approximated by sums of sin and cos terms for large enough K.
- $K \leq m/2$
- m can be non-integer
- Particularly useful for large m.

Example

- Because there are multiple seasonalities, we need to add Fourier terms for each seasonal period. In this case, the seasonal periods are 169 and 845, so the Fourier terms are of the form

$$\sin\left(\frac{2\pi kt}{169}\right), \quad \cos\left(\frac{2\pi kt}{169}\right), \quad \sin\left(\frac{2\pi kt}{845}\right), \quad \text{and} \quad \cos\left(\frac{2\pi kt}{845}\right),$$

We will fit a dynamic harmonic regression model with an ARIMA error structure.
The total number of Fourier terms for each seasonal period could be selected to minimise the AICc. However, for high seasonal periods, this tends to over-estimate the number of terms required, so we will use a more subjective choice with 10 terms for the daily seasonality and 5 for the weekly seasonality. Again, we will use a square root transformation to ensure the forecasts and prediction intervals remain positive. We set D=d=0 in order to handle the non-stationarity through the regression terms, and P=Q=0 in order to handle the seasonality through the regression terms.

Fourier Series for Seasonality in Time Series Forecasting

- The **Fourier series** is a powerful mathematical tool used to represent periodic functions as a sum of simple sine and cosine waves. In time series analysis, it is widely used to model and forecast **seasonality**, especially when the seasonality involves multiple periodic cycles with complex patterns.

Key Concept

- The idea behind Fourier series is that any periodic function can be approximated as a sum of sine and cosine functions, each representing a different harmonic (frequency). In time series forecasting, this helps to capture repeating patterns (seasonality) over different time scales such as daily, weekly, monthly, or yearly cycles.

Fourier Series Representation

For a time series y_t, a Fourier series decomposition expresses y_t as the sum of sine and cosine terms:

$$y_t = \alpha_0 + \sum_{k=1}^{K} \left[\alpha_k \cos\left(\frac{2\pi k t}{m}\right) + \beta_k \sin\left(\frac{2\pi k t}{m}\right) \right] + \epsilon_t$$

Where:

- y_t = the value of the time series at time t.
- α_0 = the intercept or mean level of the time series.
- K = the number of harmonics (frequency terms) used to approximate the seasonality.
- m = the periodicity of the seasonal cycle (e.g., 365 for yearly, 7 for weekly).
- α_k and β_k = coefficients that are estimated from the data for the cosine and sine terms, respectively.
- ϵ_t = residual error (noise not captured by the model).

How Fourier Series Models Seasonality

1. **Single Frequency (Simple Seasonality)**

In the case of simple seasonality, such as yearly seasonality in daily data, the periodicity is known (e.g., 365 days). The Fourier series can model this with a few sine and cosine terms, where the frequency m corresponds to the period of one year.

For instance, to capture yearly seasonality with $m = 365$, the seasonal component is:

$$S(t) = \alpha_1 \cos\left(\frac{2\pi t}{365}\right) + \beta_1 \sin\left(\frac{2\pi t}{365}\right)$$

This represents one complete cycle of seasonal variation, repeating every 365 time periods (days).

2. **Multiple Frequencies (Complex Seasonality)**

Many real-world time series exhibit multiple seasonal cycles simultaneously, such as daily, weekly, and yearly seasonality. Fourier series are particularly useful in such cases because it can capture multiple periodicities by adding more harmonic terms.

For instance, if a time series has both daily and yearly seasonality, the Fourier series would include terms for both frequencies:

- For **daily seasonality**:

$$S_{\text{daily}}(t) = \alpha_1 \cos\left(\frac{2\pi t}{24}\right) + \beta_1 \sin\left(\frac{2\pi t}{24}\right)$$

  where $m = 24$ (for 24 hours in a day).

- For **yearly seasonality**:

$$S_{\text{yearly}}(t) = \alpha_2 \cos\left(\frac{2\pi t}{365}\right) + \beta_2 \sin\left(\frac{2\pi t}{365}\right)$$

  where $m = 365$ (for 365 days in a year).

The resulting model would then be a sum of these two seasonal components:

$$S(t) = S_{\text{daily}}(t) + S_{\text{yearly}}(t)$$

Number of Harmonics and Approximation Accuracy

The number of harmonic terms K controls how well the Fourier series approximates the seasonal pattern. A higher K allows for capturing more complex seasonal variations.

For example, if $K = 3$, the series would include three harmonic terms for both sine and cosine:

$$S(t) = \sum_{k=1}^{3} \left[\alpha_k \cos\left(\frac{2\pi kt}{m}\right) + \beta_k \sin\left(\frac{2\pi kt}{m}\right) \right]$$

The choice of K should balance between capturing the necessary complexity of the seasonal pattern and avoiding overfitting. If K is too large, the model may overfit to noise in the data.

Fitting the Fourier Coefficients

The coefficients α_k and β_k are estimated by fitting the model to the observed data. This is typically done using **least squares regression**, where the aim is to minimize the sum of squared differences between the observed values y_t and the fitted values from the Fourier series.

Objective Function for Fitting

$$\text{Minimize} \sum_{t=1}^{N} \left(y_t - \left(\alpha_0 + \sum_{k=1}^{K} \left[\alpha_k \cos\left(\frac{2\pi kt}{m}\right) + \beta_k \sin\left(\frac{2\pi kt}{m}\right) \right] \right) \right)^2$$

Where N is the total number of observations in the time series. The fitting process adjusts the coefficients α_k and β_k to best match the observed data.

Examples of Fourier Series in Action

- **Electricity Demand Data**: Electricity consumption shows daily seasonality (due to human activity patterns), weekly seasonality (weekdays vs. weekends), and yearly seasonality (summer vs. winter). A Fourier series can capture all these periodicities by including terms for each frequency.
- **Traffic Data**: Traffic volume often shows complex seasonal patterns, with variations at different times of day (morning, afternoon), day of the week (weekday vs. weekend), and time of year (holiday season vs. regular). A Fourier series with multiple harmonics can model this complex seasonality effectively.

Fourier Series with ARIMA or Regression Models

Fourier terms can be incorporated into **ARIMA** or **regression models** to model seasonality within the context of autoregressive or regression-based time series forecasting.

For example, an ARIMA model with Fourier terms for seasonal components can be written as:

$$y_t = \sum_{k=1}^{K} \left[\alpha_k \cos\left(\frac{2\pi k t}{m}\right) + \beta_k \sin\left(\frac{2\pi k t}{m}\right) \right] + \sum_{i=1}^{p} \phi_i y_{t-i} + \sum_{j=1}^{q} \theta_j \epsilon_{t-j} + \epsilon_t$$

Where the Fourier terms capture seasonality and the ARIMA part captures non-seasonal patterns and noise.

Prophet model

- The **Prophet model**, developed by Facebook, is a time series forecasting tool specifically designed to handle datasets with multiple seasonality's, holidays, and trends. Prophet is an additive model, making it intuitive and flexible for various types of time series data, including those with missing data or outliers. One of its key strengths is the ability to handle daily, weekly, and yearly seasonality, along with holidays and special events, in a simple and interpretable way.

Key Features of the Prophet Model

- **Piecewise Linear or Logistic Trend**: Prophet can model both linear and logistic growth, allowing for flexible handling of different types of trends.
- **Multiple Seasonalities**: It supports multiple seasonal patterns, such as daily, weekly, and yearly cycles.
- **Holiday Effects**: The model allows the inclusion of custom holiday and event effects to account for spikes or dips in the time series.
- **Outlier Robustness**: Prophet is designed to handle missing data and outliers in a robust manner.
- **Automatic Hyperparameter Tuning**: Many parameters are automatically tuned, reducing the need for manual intervention, but the model also provides flexibility for customization.

- A recent proposal is the Prophet model, available via the `fable.prophet` package. This model was introduced by Facebook (S. J. Taylor & Letham, 2018), originally for forecasting daily data with weekly and yearly seasonality, plus holiday effects. It was later extended to cover more types of seasonal data. It works best with time series that have strong seasonality and several seasons of historical data.
- Prophet can be considered a nonlinear regression model (Chapter 7), of the form

$$y_t = g(t) + s(t) + h(t) + \varepsilon_t,$$

where $g(t)$ describes a piecewise-linear trend (or "growth term"), $s(t)$ describes the various seasonal patterns, $h(t)$ captures the holiday effects, and ε_t is a white noise error term.

- The knots (or changepoints) for the piecewise-linear trend are automatically selected if not explicitly specified. Optionally, a logistic function can be used to set an upper bound on the trend.
- The seasonal component consists of Fourier terms of the relevant periods. By default, order 10 is used for annual seasonality and order 3 is used for weekly seasonality.
- Holiday effects are added as simple dummy variables.
- The model is estimated using a Bayesian approach to allow for automatic selection of the changepoints and other model characteristics.

1. **Trend Component ($g(t)$)**

The trend component models the long-term increase or decrease in the data. Prophet supports two types of trends: **piecewise linear** and **logistic growth**.

a. **Piecewise Linear Trend**

This is the default trend model in Prophet. It assumes the time series grows linearly, with potential changes in the slope at specific points in time (changepoints).

The piecewise linear trend is given by:

$$g(t) = (k + a(t)^T \delta)t + (m + a(t)^T \gamma)$$

Where:

- k is the initial rate of growth (slope of the trend before any changepoints).
- m is the initial offset (intercept) of the trend.
- t is the time.
- $a(t)$ is an indicator function that selects the appropriate changepoint, defined as:

- $a(t)$ is an indicator function that selects the appropriate changepoint, defined as:

$$a(t) = \begin{cases} 1 & \text{if } t \geq \tau_j \\ 0 & \text{otherwise} \end{cases}$$

- δ is the vector of changepoints where the slope changes.
- γ is the vector of adjustments made to the intercept after changepoints.

The trend allows the slope to change at pre-determined changepoints τ_j, providing flexibility in modeling non-linear trends.

b. **Logistic Growth Trend**

The logistic growth trend models a time series that grows exponentially but slows down as it approaches a carrying capacity, which represents a maximum possible value.

The logistic growth function is:

$$g(t) = \frac{C}{1 + \exp(-k(t - m))}$$

Where:

- C is the **carrying capacity**, or the maximum value the time series can reach.
- k is the **growth rate**, determining how fast the series approaches the carrying capacity.
- m is the **midpoint**, the time at which the growth is halfway to the carrying capacity.

2. Seasonality Component ($s(t)$)

Seasonality in Prophet is modeled using Fourier series, which allow the model to capture periodic patterns in the data. The seasonality is expressed as a sum of sine and cosine terms.

For a seasonal period P (e.g., yearly or weekly seasonality), the seasonal component $s(t)$ is modeled as:

$$s(t) = \sum_{n=1}^{N} \left(a_n \cos\left(\frac{2\pi n t}{P}\right) + b_n \sin\left(\frac{2\pi n t}{P}\right) \right)$$

Where:

- N is the number of Fourier terms used to model the seasonality (higher N captures more complex seasonal patterns).
- a_n and b_n are the coefficients of the Fourier terms that are learned during the fitting process.
- P is the period of the seasonality (e.g., 365.25 for yearly seasonality or 7 for weekly seasonality).

Prophet includes predefined seasonality for:

- **Yearly seasonality** with period $P = 365.25$.
- **Weekly seasonality** with period $P = 7$.
- **Daily seasonality** (for hourly data).

3. **Holiday Component ($h(t)$)**

The holiday component allows the model to capture special events or holidays that cause predictable deviations from the trend. This is modeled as an additional piecewise linear term that adds a fixed effect during holiday periods.

The holiday effect is modeled as:

$$h(t) = \sum_{i=1}^{H} \alpha_i \mathbf{1}_{[t \in \text{holiday}_i]}$$

Where:

- H is the total number of holidays.
- α_i is the effect of holiday i, learned from the data.
- $\mathbf{1}_{[t \in \text{holiday}_i]}$ is an indicator function that is 1 if time t falls within the holiday period i, and 0 otherwise.

4. Error Term (ϵ_t)

The error term represents the noise or residual component, which is the part of the time series that cannot be explained by the trend, seasonality, or holidays. This term is assumed to be normally distributed:

$$\epsilon_t \sim \mathcal{N}(0, \sigma^2)$$

Where:

- $\mathcal{N}(0, \sigma^2)$ represents a normal distribution with mean 0 and variance σ^2.

Vector autoregressions

- One limitation of the models that we have considered so far is that they impose a unidirectional relationship — the forecast variable is influenced by the predictor variables, but not vice versa.
- However, there are many cases where the reverse should also be allowed for — where all variables affect each other.
- Example - the changes in personal consumption expenditure (Ct) were forecast based on the changes in personal disposable income (It).
- However, in this case a bi-directional relationship may be more suitable: an increase in It will lead to an increase in Ct and vice versa.

Vector autoregressions

- An example of such a situation occurred in Australia during the Global Financial Crisis of 2008–2009. The Australian government issued stimulus packages that included cash payments in December 2008, just in time for Christmas spending. As a result, retailers reported strong sales and the economy was stimulated. Consequently, incomes increased.
- Such feedback relationships are allowed for in the vector autoregressive (VAR) framework. In this framework, all variables are treated symmetrically. They are all modelled as if they all influence each other equally.

Vector autoregressions

- A VAR model is a generalization of the univariate autoregressive model for forecasting a vector of time series. It comprises one equation per variable in the system. The right-hand side of each equation includes a constant and lags of all of the variables in the system. To keep it simple, we will consider a two variable VAR with one lag. We write a 2-dimensional VAR(1) model as

$$y_{1,t} = c_1 + \phi_{11,1} y_{1,t-1} + \phi_{12,1} y_{2,t-1} + \varepsilon_{1,t}$$
$$y_{2,t} = c_2 + \phi_{21,1} y_{1,t-1} + \phi_{22,1} y_{2,t-1} + \varepsilon_{2,t},$$

where $\varepsilon_{1,t}$ and $\varepsilon_{2,t}$ are white noise processes that may be contemporaneously correlated. The coefficient $\phi_{ii,\ell}$ captures the influence of the ℓth lag of variable y_i on itself, while the coefficient $\phi_{ij,\ell}$ captures the influence of the ℓth lag of variable y_j on y_i.

Vector autoregressions

- Forecasts are generated from a VAR in a recursive manner. The VAR generates forecasts for *each* variable included in the system. To illustrate the process, assume that we have fitted the 2-dimensional VAR(1) model described for all observations up to time T. Then the one-step-ahead forecasts are generated by

$$\hat{y}_{1,T+1|T} = \hat{c}_1 + \hat{\phi}_{11,1} y_{1,T} + \hat{\phi}_{12,1} y_{2,T}$$
$$\hat{y}_{2,T+1|T} = \hat{c}_2 + \hat{\phi}_{21,1} y_{1,T} + \hat{\phi}_{22,1} y_{2,T}.$$

Vector autoregressions

- For h=2, the forecasts are given by

$$\hat{y}_{1,T+2|T} = \hat{c}_1 + \hat{\phi}_{11,1}\hat{y}_{1,T+1|T} + \hat{\phi}_{12,1}\hat{y}_{2,T+1|T}$$
$$\hat{y}_{2,T+2|T} = \hat{c}_2 + \hat{\phi}_{21,1}\hat{y}_{1,T+1|T} + \hat{\phi}_{22,1}\hat{y}_{2,T+1|T}.$$

this is the same as above VAR equations , except that the errors have been set to zero, the parameters have been replaced with their estimates, and the unknown values of y1 and y2 have been replaced with their forecasts. The process can be iterated in this manner for all future time periods.

Vector autoregressions- Number of Parameters to be estimated

- There are two decisions one has to make when using a VAR to forecast, namely how many variables (denoted by K) and how many lags (denoted by p) should be included in the system.
- The number of coefficients to be estimated in a VAR is equal to K+pK^2 (1+pK per equation).
- For example, for a VAR with K=5 variables and p=3 lags, there are 16 coefficients per equation, giving a total of 80 coefficients to be estimated.
- The more coefficients that need to be estimated, the larger the estimation error entering the forecast.

Vector autoregressions- Number of Parameters to be estimated

- In practice, it is usual to keep K small and include only variables that are correlated with each other, and therefore useful in forecasting each other.
- Information criteria are commonly used to select the number of lags to be included.
- Care should be taken when using the AICc as it tends to choose large numbers of lags; instead, for VAR models, we often use the BIC instead.
- A more sophisticated version of the model is a “sparse VAR” (where many coefficients are set to zero); another approach is to use “shrinkage estimation” (where coefficients are smaller).

Application

VARs are useful in several contexts:

- forecasting a collection of related variables where no explicit interpretation is required;
- testing whether one variable is useful in forecasting another (the basis of Granger causality tests);
- impulse response analysis, where the response of one variable to a sudden but temporary change in another variable is analysed;
- forecast error variance decomposition, where the proportion of the forecast variance of each variable is attributed to the effects of the other variables.

Bootstrapping and bagging

- When a normal distribution for the residuals is an unreasonable assumption, one alternative is to use bootstrapping, which only assumes that the residuals are uncorrelated with constant variance.
- A one-step forecast error is defined as

$$e_t = y_t - \hat{y}_{t|t-1}$$

For a naïve forecasting method

$$y_t = y_{t-1} + e_t.$$

- Assuming future errors will be similar to past errors, when t>T we can replace et by sampling from the collection of errors we have seen in the past (i.e., the residuals). So we can simulate the next observation of a time series using

$$y^*_{T+1} = y_T + e^*_{T+1}$$

Bootstrapping and bagging

- Adding the new simulated observation to our data set, we can repeat the process to obtain

$$y^*_{T+2} = y^*_{T+1} + e^*_{T+2},$$

where e∗T+2is another draw from the collection of residuals. Continuing in this way, we can simulate an entire set of future values for our time series.

Example


Google daily closing stock price
800
700
600
500
$US
0
100
200
day

Bootstrapping time series

- More generally, we can generate new time series that are similar to our observed series, using another type of bootstrap.
- First, the time series is transformed if necessary, and then decomposed into trend, seasonal and remainder components using STL.
- Then we obtain shuffled versions of the remainder component to get bootstrapped remainder series.
- Because there may be autocorrelation present in an STL remainder series, we cannot simply use the re-draw procedure that was described above .

Bootstrapping time series

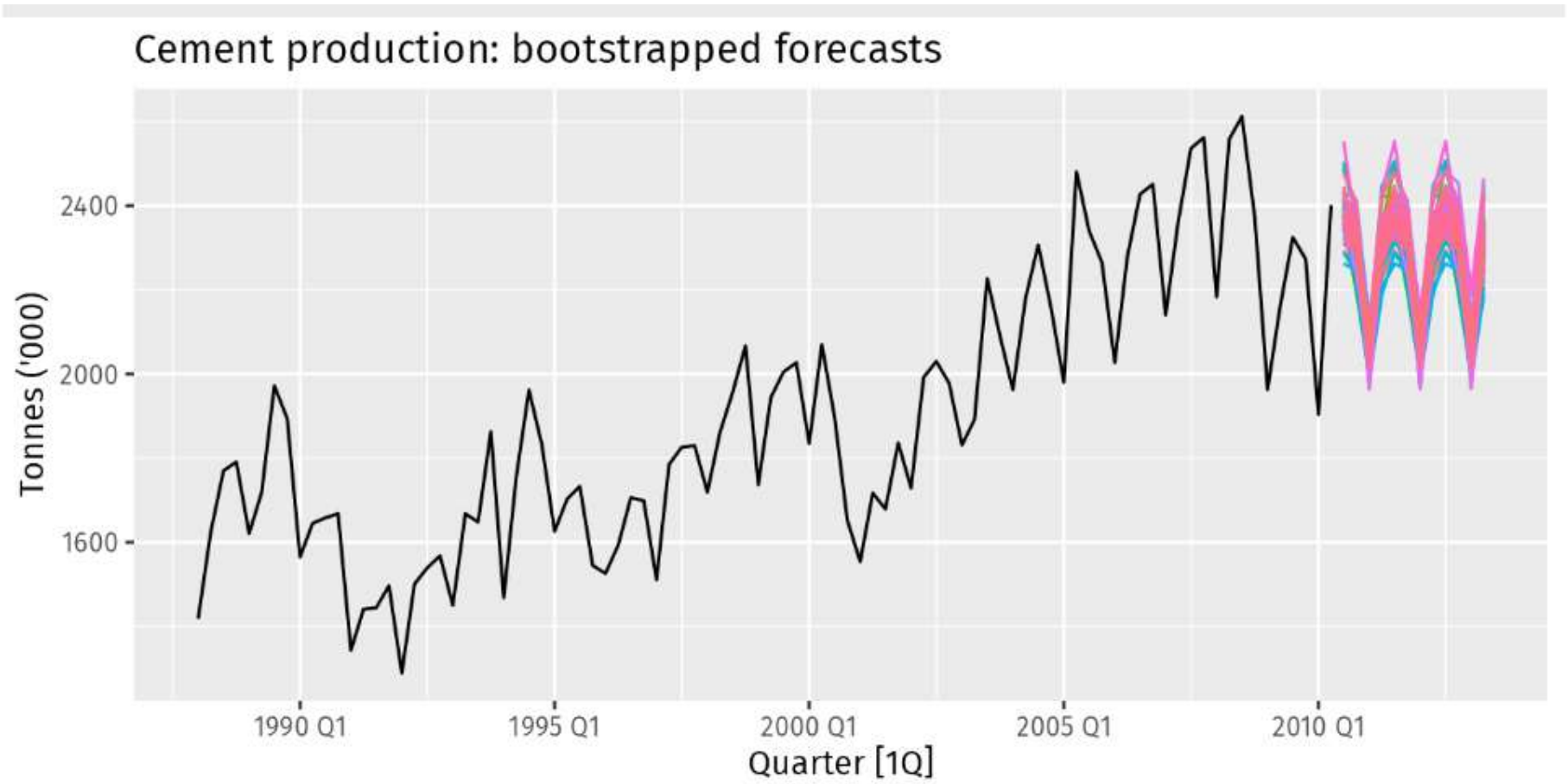
- Instead, we use a “blocked bootstrap”, where contiguous sections of the time series are selected at random and joined together.
- These bootstrapped remainder series are added to the trend and seasonal components, and the transformation is reversed to give variations on the original time series.

Bagged forecasts

- One use for these bootstrapped time series is to improve forecast accuracy. If we produce forecasts from each of the additional time series, and average the resulting forecasts, we get better forecasts than if we simply forecast the original time series directly. This is called “bagging” which stands for “**b**ootstrap **agg**regatin**g**”.
- For each of these series, we fit an ETS model. A different ETS model may be selected in each case, although it will most likely select the same model because the series are similar. However, the estimated parameters will be different, so the forecasts will be different even if the selected model is the same. This is a time-consuming process as there are a large number of series.

Example

We demonstrate the idea using the cement data. First, we simulate many time series that are similar to the original data, using the block-bootstrap described above.



Example

- Finally, we average these forecasts for each time period to obtain the “bagged forecasts” for the original data.

Fig show that, on average, bagging gives better forecasts than just applying ETS() directly. Of course, it is slower because a lot more computation is required.